

A Report on Market Program in C++ Using Object Oriented Programming

Bruno Bucalon Serra
Department of Computer Science
University of London

CONTENTS

I	Introduction	1
II	Task Discussion	1
II-A	TASK 1: Loading Trading Data and Computing Candlestick Data	1
II-B	TASK 2: Login or Register a Trading User profile	2
II-C	TASK 3: User Wallet and Trading Transactions History	2
II-D	TASK 4: Simulating User Trading Activities	3
II-E	TASK 5: Persistent Audit Logging and Wallet Recovery	3
III	Conclusion	3
	References	3

LIST OF TABLES

I	Summary of Tasks and Corresponding Code Evidence	1
---	--	---

LIST OF FIGURES

1	Unified Modeling Language - UML	1
2	Candlestick output.	2
3	Login screen.	2
4	Insufficient funds.	2
5	Creating orders outside the date range.	3
6	Loaded orders.	4
7	Trade history	4

A Report on Market Program in C++ Using Object Oriented Programming

TABLE I. SUMMARY OF TASKS AND CORRESPONDING CODE EVIDENCE

Task	Feature	Code File	CODE PDF Pages
Task 1	Candlestick computation	OrderBook.cpp, Candlestick.h, MerkelMain.cpp	pp. 11–12, 30–31
Task 2	User login and registration	MerkelMain.cpp	pp. 12–15
Task 3	Wallet validation and trades	Wallet.h, MerkelMain.cpp	pp. 8–10
Task 4	Trading simulation	MerkelMain.cpp	pp. 18–19
Task 5	Wallet persistence	MerkelMain.cpp	pp. 15–20

Abstract—This report presents an analysis of the development of a project using object oriented programming in C++ for the Object Oriented Programming module of the BSc Computer Science programme at the University of London, with Goldsmiths direction. It contains a suitable explanation, implementation logic discussion (with fragments of pseudocode where relevant) and a screenshot and evidence of the final output of the entire project.

I. INTRODUCTION

The object oriented programming (OOP) is a programming paradigm that structures and organizes the code lines of a project around objects, instead of functions and traditional control. The objects are entities that represents elements from the real world of abstractions, having characteristics (attributes) and specific associated behaviors (methods).

The goal of this project is developing a market program for bid and ask transactions from scratch using C++ programming language, focused on five main tasks: loading trading data and computing candlestick data, login or register a trading user profile, user wallet and trading transaction history, simulating user trading activities, and printing user interaction menu to support user login and trading operations. Printing user interaction menu to support user login and trading operations.

II. TASK DISCUSSION

In this section, all tasks mentioned in the introduction will be discussed with a suitable explanation, implementation logic discussion (with fragments of pseudocode where relevant) and a screenshot and evidence of the final output.

A. TASK 1: Loading Trading Data and Computing Candlestick Data

Explanation of this task involves loading trading orders from the provided CSV file and creating a summary table for a particular product. The user should be able to enter a product

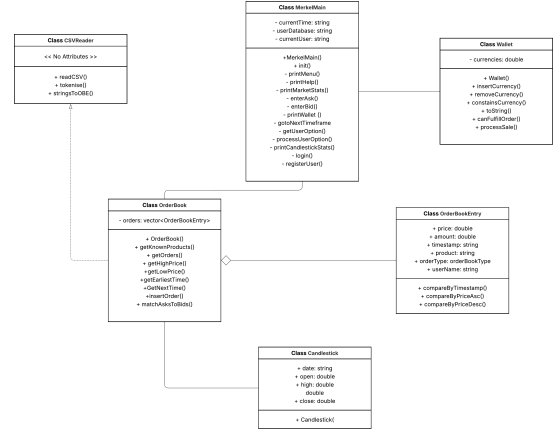


Fig. 1. Unified Modeling Language - UML

combination, i.e., ‘ETH/USDT’ and by default yearly summary should be displayed and should also be able to specify and filter the product’s record by date range (i.e., daily, monthly or yearly).

The implementation logic this feature involved creating a new class Candlestick was encapsulated into the program with a static function in OrderBook, where iterates thorough the filtered OrderBookEntries by product and time.

Pseudocode:

```
// Logic for computing OHLC from entries
```

```
high = -infinity, low = +infinity
open = orders[0].price
close = orders[last].price
```

```
For each order in filtered_orders:
```

```
    if order.price>high: high = order.price
    if order.price<low: low = order.price
```

```
return Candlestick(open, high, low, close)
```

Candlestick (OHLC) computation is implemented in `OrderBook::computeCandlesticks()` (CODE PDF pp. 30–31), with user interaction and output handled in `MerkelMain::printCandlestickStats()` (CODE PDF pp. 11–12).

The final output as evidence is rendered as a formatted table displaying the calculated values for the requested product and timestamp, as shown in Fig 1.

```

Current time is: 2020/06/01 11:57:30.328127
Type in 1-6
7
You chose: 7
Select Product (e.g.: ETH/BTC):
ETH/BTC
Enter Date (YYYY/MM/DD HH:MM:SS) or press Enter for current time:

=== CANDLESTICK for ETH/BTC ===
Date: 2020/06/01 11:57:30.328127
Open: 0.0248261
High: 0.0251581
Low: 0.0248261
Close: 0.0251581

```

Fig. 2. Candlestick output.

B. TASK 2: Login or Register a Trading User profile

This task is explained by allowing users to register or log in with their existing username and password. It will add flexibility to the analytics system, allowing users to filter data from any timestamp, including generic data.

Implementation Logic relies on a string-comparison mechanism within the CSVReader and OrderBook classes. By treating timestamps as strings, the filter can perform exact matches against the data stored in memory. The logic was improved to handle user-defined inputs from the terminal, converting them into query parameters that the `getOrders` function uses to cut the dataset.

```

FUNCTION getFilteredOrders(targetProduct,
targetTime):
    filteredVector = empty list

    FOR EACH entry IN globalOrderBook:
        IF entry.product == targetProduct
        AND entry.timestamp == targetTime:
            APPEND entry TO filteredVector

    IF filteredVector IS EMPTY:
        PRINT "No data found for
        the specified timeframe"

    RETURN filteredVector

```

User registration, authentication, and credential loading are implemented in `MerkelMain::registerUser()`, `MerkelMain::login()`, and `MerkelMain::loadUsers()` respectively (CODE PDF pp. 12–15).

```

1: Login
2: Register
Type in 1 or 2.
1
You chose: 1
--- LOGIN ---
userId:
98506201
Password: 123456
Wallet loaded from persistent storage.
Welcome bruno serra!

```

Fig. 3. Login screen.

C. TASK 3: User Wallet and Trading Transactions History

Explanation for the wallet management system centers on the improvement of financial integrity within the exchange. Before any Bid or Ask can be successfully added into the OrderBook, the system must perform validations of the user's available assets. The system will manage multiple currencies simultaneously.

Implementation Logic utilizes a `std::map<std::string, double>` to store currency-quantity pairs, providing $O(\log n)$ lookup times. The `canFulfillOrder` function acts as a gatekeeper, calculating the total cost of a Bid or the required amount of a commodity for an Ask. If the balance check passes, the transaction is authorized; otherwise, an exception is handled, and the user is notified of insufficient funds. Furthermore, the `processSale` method handles the post-trade settlement, updating balances for both the base and quote currencies.

Pseudocode:

```

FUNCTION getFilteredOrders(targetProduct,
targetTime):

    filteredVector = empty list

    FOR EACH entry IN globalOrderBook:

        IF entry.product == targetProduct
        AND entry.timestamp == targetTime:
            APPEND entry TO filteredVector

    IF filteredVector IS EMPTY:
        PRINT "No data found for
        the specified timeframe"

    RETURN filteredVector

```

Wallet balance validation and post-trade updates are handled via `Wallet::canFulfillOrder()` and `MerkelMain::gotoNextTimeframe()` (CODE PDF pp. 8–10).

```

1: Print help
2: Print exchange stats
3: Make an offer
4: Make a bid
5: Print wallet
6: Continue
7: View Candlestick Stats
8: Deposit money
9: Withdraw money
10: Print trade history.
11: Print trading stats.
12: Simulate Trading.
=====
Current time is: 2020/06/01 11:57:30.328127
Type in 1-6
4
You chose: 4
Mark an bid - enter the amount: product, price, amount, eg <ETH/BTC, 200, 0.5>
ETH/BTC, 100000, 0.5
Wallet::canFulfillOrder BTC : 50000
Wallet has insufficient funds .

```

Fig. 4. Insufficient funds.

D. TASK 4: Simulating User Trading Activities

Explanation for the trading simulator involves the creation of an engine designed to test the exchange's logic. The simulator updates the market with a high volume of orders across all available products, ensuring that the OrderBook is never stagnant.

Implementation logic is built upon a nested iteration over the list of known products, where the simulator acts as an automated "market maker." For each product, it generates five distinct OrderBookEntry objects with varying price points. One aspect of this implementation that makes it unique is its dual-timestamping capability: it uses both the current system time and a user-selected retroactive time, allowing the simulator to "fill in" historical gaps in the dataset.

Pseudocode:

```
FUNCTION simulateTrading(backdatedTime,
realTime):
    products = orderBook.getKnownProducts()

    FOR EACH p IN products:
        FOR i FROM 1 TO 5:
            // Alternate between real
            and backdated time for diversity
            selectedTime = (i % 2 == 0) ?
            realTime: backdatedTime

            price = basePrice + i

            newOrder = CreateOrder(price, 0.5,
            selectedTime, p, TYPE_ASK)

            orderBook.insert(newOrder)

            saveToCSV("new_bid_ask.csv",
            newOrder)

            wallet.removeCurrency(p.baseCurrency,
            0.5)
```

Trading simulation logic is implemented in `MerkelMain::simulateTrading()` (CODE PDF pp. 18–19), generating synthetic market orders and persisting simulated activity.

E. TASK 5: Persistent Audit Logging and Wallet Recovery

Explanation of this task is that, in a real-world scenario, a user's session must be recoverable. This task implements an audit log of all financial activities, including deposits, withdrawals, and simulated trades. By moving data from volatile memory (RAM) to persistent storage (CSV), the system ensures that the user's financial state is maintained even after the application is closed and restarted.

Implementation Logic centers on two phases: persistent writing and initialization-loading. Every financial operation triggers a write event to `wallet_activity.csv`, recording the User ID, operation type, currency, amount, and the re-

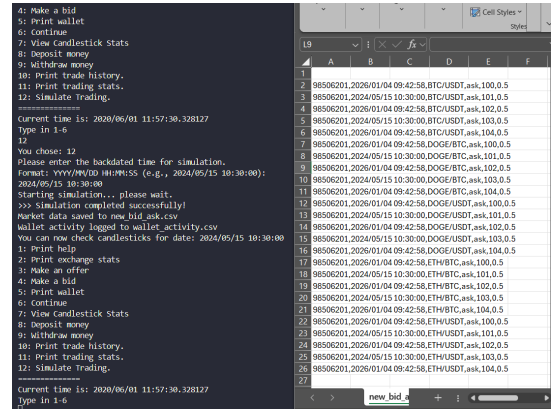


Fig. 5. Creating orders outside the date range.

sulting balance. Then, the `loadUserWallet` function parses this file, filtering entries by the `currentUser ID`.

Pseudocode:

```
FUNCTION loadUserWallet(userID):
    FOR EACH line IN wallet_activity.csv:
        tokens = parse(line)

        IF tokens.id == userID:
            IF tokens.type == "deposit":
                wallet.add(tokens.currency,
                tokens.amount)

            IF tokens.type == "withdraw"
            OR "simulated_trade":
                wallet.remove(tokens.currency,
                tokens.amount)
```

Persistent wallet logging and recovery are implemented in `MerkelMain::depositMoney()`, `MerkelMain::withdrawMoney()`, and `MerkelMain::loadUserWallet()` (CODE PDF pp. 15–20).

III. CONCLUSION

The project of market exchange platform was successfully integrated with a financial security. By implementing a modular architecture, the system efficiently manages large-scale datasets through the Candlestick aggregation engine while maintaining integrity via the Wallet validation layer. In summary, the project demonstrates principles of C++ applied with object oriented programming.

Furthermore, managing object-oriented principles (OOP) was difficult due to memory management, since it was the first time I was able to use a low level programming language. However, the project was properly completed with all tasks working as it should.

REFERENCES

- [1] Lafore, R., Object-Oriented Programming in C++, Addison-Wesley, 1st ed., December 19, 2001. 01.

```

CSVReader::readCSV read 1021772 entries
1: Login
2: Register
Type in 1 or 2.
1
You chose: 1
--- LOGIN ---
userId:
98506201
Password: 23
Invalid user ID or password. Try again.
1: Login
2: Register
Type in 1 or 2.
1
You chose: 1
--- LOGIN ---
userId:
98506201
Password: 123456
Wallet loaded from persistent storage.
Welcome bruno serra!

```

Fig. 6. Loaded orders.

```

Type in 1-6
10
You chose: 10
--- Last 5 transactions ---
2020/06/01 11:57:35.334211,98506201,ETH/BTC,bid,0.0248467,15
2020/06/01 11:57:35.334211,98506201,ETH/BTC,bid,0.0248467,14.5
1: Print help
2: Print exchange stats
3: Make an offer
4: Make a bid
5: Print wallet
6: Continue
7: View Candlestick Stats
8: Deposit money
9: Withdraw money
10: Print trade history.
11: Print trading stats.
12: Simulate Trading.
=====
Current time is: 2020/06/01 11:57:30.328127
Type in 1-6

```

Fig. 7. Trade history